T05 Ranged Weapons

CRE132 worksheet

What you'll make

A fireball you aim with the mouse. Point at a slime and hold the right mouse button: fireballs fly at the pointer, four a second, in any direction. Each hit turns a slime redder, and ten remove it. Fireballs burst on tree trunks, rocks and fences, so the slime behind the fence is safe until you walk round it.

By the end you'll know the three steps of mouse aiming (pointer in pixels, to a world position, to a direction), and why a top-down projectile flies along the ground with its picture drawn above.



Figure 1: The finished firing range: the player on a patch of dirt, a rock, a tree, a fence and two slimes.

Before you start

1. Unzip `CRE132_T05_RangedWeapons_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T05_RangedWeapons_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T05_RangedWeapons`.
3. Press **Play**. You can walk and swing T04's sword (J or the left mouse button), but the right mouse button does nothing yet. Press Play again to stop.

The scene has T04's **Player**, five **Slime Dummies** with 10 health, and two S07 pools under **Pools**: **Fireball Pool** (10 of `Fireball (Top-Down)`) and **Impact Pool** (5 of `Fireball Impact (Top-Down)`). You'll write the fireball and the shooter.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Two input actions: Fire and Aim

Double-click `CRE132Controls` (in `Assets/_game/Input`) and add two actions to the **Player** map:

- **Fire:** click **+** next to **Actions**, name it `Fire` and leave **Action Type** as **Button**. Give it three bindings: **Right Button [Mouse]**, **K [Keyboard]** and **Right Trigger [Gamepad]**. Set the first binding's **Path**, then use **+ > Add Binding** for the other two.
- **Aim:** add an action called `Aim` with **Action Type Value** and **Control Type Vector2**, like Move. Set its binding's Path to **Position [Pointer]**: open **Pointer** in the Path list and pick **Position** (**Listen** only finds buttons). "Pointer" covers the mouse, a pen and touch.

Click **Save Asset** at the top of the window (or tick **Auto-Save**) and close the editor.

Aim gives the pointer's position **on the screen, in pixels**: (0, 0) is the bottom-left corner of the Game view. The game world uses units, not pixels, so the shooter has to convert it.

2. The fireball: AimedProjectile

Make a folder `Assets/_game/Scripts/T05_RangedWeapons` and create `AimedProjectile` in it:

```
// S07's fireball, flying in any direction instead of only left or right. The root (with
// the trigger collider) moves along the ground; the Sprite child is drawn higher up and
// turned to face the way it flies. Hurts anything with Health, then shows an impact and
// goes back to its pool.
using UnityEngine;

public class AimedProjectile : MonoBehaviour
{
    [SerializeField] private float speed = 10f;
    [SerializeField] private float lifetime = 1.2f;
    [SerializeField] private int damage = 1;
    // The child with the SpriteRenderer.
    [SerializeField] private Transform spriteTransform;

    private Rigidbody2D body;
    private ObjectPool impactPool;
    private float timeLeft;
    private bool flyingLeft;

    private void Awake()
    {
        body = GetComponent<Rigidbody2D>();
    }

    // Called by the shooter each time the fireball comes out of the pool. A reused object
    // still has the values from its last flight, so everything that changes per shot is
    // set again here.
    public void Launch(Vector2 direction, ObjectPool newImpactPool)
    {
        impactPool = newImpactPool;
        timeLeft = lifetime;
        body.linearVelocity = direction * speed;

        // The fireball picture points right. Setting the sprite's "right" to the
        // direction turns it to match.
        spriteTransform.right = direction;
```

```
        flyingLeft = direction.x < 0f;
    }

    private void Update()
    {
        // Missed everything: give it back to the pool instead of destroying it.
        timeLeft = timeLeft - Time.deltaTime;
        if (timeLeft <= 0f)
        {
            gameObject.SetActive(false);
        }
    }

    private void OnTriggerEnter2D(Collider2D other)
    {
        // Fly through the player who fired it and through other triggers (the sword
        // hitbox, pickups).
        if (other.isTrigger || other.CompareTag("Player"))
        {
            return;
        }

        Health health = other.GetComponent<Health>();
        if (health != null)
        {
            health.TakeDamage(damage);
        }

        // The impact appears where the picture is, not down at the shadow.
        GameObject impact = impactPool.Get(spriteTransform.position);
        impact.GetComponent<SpriteRenderer>().flipX = flyingLeft;

        gameObject.SetActive(false);
    }
}
```

What's going on:

- **`Launch` replaces `Start`.** A pooled fireball is reused, so it still has the values from its last flight. The shooter calls `Launch` each time, which resets the timer, the velocity and the direction.
- **`direction * speed`**: the direction is always 1 long, so every shot has the same speed whichever way it goes.
- **`spriteTransform.right = direction`** turns the picture. The art points right, so pointing its "right" along the direction turns it to match. Only the child turns, so the collider stays where it is.
- **`OnTriggerEnter2D` skips other triggers and the Player**, hurts anything with `Health`, takes an impact from the pool and switches the fireball off, which puts it back in the pool.
- **The impact appears at `spriteTransform.position`**, where the picture is, not down at the root.

Add it: open `Assets/_game/Prefabs/Projectiles/Fireball (Top-Down)` (double-click). Select the root, **Fireball (Top-Down)**, and add **Aimed Projectile**. Drag its child **Sprite** into the **Sprite Transform** slot. Leave **Speed** 10, **Lifetime** 1.2 and **Damage** 1. Leave the prefab with the back arrow in the Hierarchy.

Look at the prefab while you're there: the root has a **Kinematic Rigidbody 2D**, a small trigger **Circle Collider 2D** (**Radius** 0.2) and a **Sorting Group**. The **Sprite** child is 0.6 higher.

3. Aiming: AimedShooter

Create `AimedShooter` in the same folder:

```
// Hold Fire (right mouse button, K, gamepad right trigger) to throw fireballs at the
// mouse pointer. Fireballs come from S07's pools instead of Instantiate. In a top-down
// game the colliders are at the feet, so the fireball flies along the ground (its
// "shadow") with its picture drawn higher up. We aim from the picture's height and fire
// from the feet, so the picture flies over the pointer and the shadow hits the target's
// feet.
using UnityEngine;
using UnityEngine.InputSystem;

public class AimedShooter : MonoBehaviour
{
    [SerializeField] private InputActionReference fireAction;
    // The pointer's position on screen, in pixels.
    [SerializeField] private InputActionReference aimAction;
    [SerializeField] private ObjectPool fireballPool;
    [SerializeField] private ObjectPool impactPool;
    [SerializeField] private float timeBetweenShots = 0.25f;

    // How high above the feet the fireball is drawn (the same as the Sprite child on the
    // Fireball prefab).
    [SerializeField] private float shotHeight = 0.6f;
    // How far in front of the player the fireball appears, so it doesn't start inside the
    // body.
    [SerializeField] private float spawnDistance = 0.5f;

    private Camera mainCamera;
    private float nextShotTime;

    private void Awake()
    {
        mainCamera = Camera.main;
    }

    private void OnEnable()
    {
        fireAction.action.Enable();
        aimAction.action.Enable();
    }

    private void OnDisable()
    {
        fireAction.action.Disable();
        aimAction.action.Disable();
    }

    private void Update()
    {
        // IsPressed (held), so holding the button keeps firing.
        if (fireAction.action.IsPressed() && Time.time >= nextShotTime)
        {
            Shoot();
            nextShotTime = Time.time + timeBetweenShots;
        }
    }

    private void Shoot()
    {
        // Screen pixels to a point in the world. Our camera is orthographic, so the Z it
        // gives back doesn't matter: we only use X and Y.
```

```
        Vector2 pointerOnScreen = aimAction.action.ReadValue<Vector2>();
        Vector2 pointerInWorld = mainCamera.ScreenToWorldPoint(pointerOnScreen);

        Vector2 feet = transform.position;
        Vector2 aimFrom = feet + new Vector2(0f, shotHeight);

        // A direction is a vector of length 1: subtract to get from A to B, then
        // normalise.
        Vector2 direction = (pointerInWorld - aimFrom).normalized;
        if (direction == Vector2.zero)
        {
            // Pointer exactly on the player: there's no direction to shoot in.
            return;
        }

        GameObject fireball = fireballPool.Get(feet + direction * spawnDistance);
        fireball.GetComponent<AimedProjectile>().Launch(direction, impactPool);
    }
}
```

What's going on:

- **`IsPressed`** is true every frame the button is held (T04 used `WasPressedThisFrame`, true only once). With the timer, holding the button fires four times a second.
- **`ScreenToWorldPoint`** turns the pointer's pixels into a position in the world.
- **`pointerInWorld - aimFrom`** gives the arrow from the player to the pointer. **`.normalized`** makes it 1 long, so only the direction is left.
- **The fireball flies along the ground, its picture above.** Colliders in a top-down game are at the feet (T03, T04), so the fireball's root and trigger fly at foot level like a shadow, with the sprite drawn 0.6 higher. The shooter aims from that height (`aimFrom`) and fires from the feet (`feet`), so the picture flies over the pointer and the "shadow" hits the slime's feet.
- **The Sorting Group on the fireball** sorts it by its root, so it passes behind slimes and trees lower on the screen, like the characters in T03.

Add it: select the **Player** (the root) and add **Aimed Shooter**. Fill in:

- **Fire Action**: **Player/Fire**. **Aim Action**: **Player/Aim**.
- **Fireball Pool**: drag **Fireball Pool** (under **Pools**) into the slot. **Impact Pool**: drag **Impact Pool**.
- Leave **Time Between Shots** 0.25, **Shot Height** 0.6 and **Spawn Distance** 0.5.

Test it: press Play, point at a slime and hold the right mouse button. Fireballs fly at the pointer and burst on the slime, which gets redder. Shoot at a tree trunk, a rock and the fence: they burst there too. Walk round the fence to reach the slime behind it.

Check your work

- ☐ `CRE132Controls` has **Fire** (Button: right mouse button, K, Right Trigger) and **Aim** (Value, Vector2: pointer position).
- ☐ Holding the right mouse button (or K) fires four fireballs a second at the pointer, in any direction.
- ☐ Each fireball is turned to face the way it flies.
- ☐ A hit turns a slime redder; ten hits remove it.
- ☐ Fireballs burst on trunks, rocks and the fence, and disappear after a short distance if they hit nothing.
- ☐ Fireballs pass behind slimes and trees that are lower on the screen.
- ☐ Compare with `CRE132_T05_RangedWeapons_Finished` if something behaves differently.

Key points

- **Mouse aiming is three steps:** pointer in pixels, to a world position, to a direction.
- **A normalised direction times a speed** gives the same speed whichever way you shoot.
- **Keep the collider at the feet and the picture above it.** That's what makes top-down projectiles hit what they appear to hit.

If something goes wrong

- **NullReferenceException in AimedShooter:** one of its four slots is empty, or the scene's camera isn't tagged **MainCamera** (`Camera.main` finds the camera by that tag).
- **NullReferenceException in AimedProjectile's Launch:** the prefab's **Sprite Transform** slot is empty.
- **NullReferenceException in Shoot:** the **Fireball (Top-Down)** prefab has no **Aimed Projectile**. Open the prefab and add it there.
- **The fireball doesn't turn:** you dragged the root into **Sprite Transform** instead of the **Sprite** child.
- **Fireballs pass a little above or below the pointer: Shot Height** must match the Sprite child's height on the prefab (0.6).

Try it

1. Make the player face the way they shoot. (Hint: T02's animator direction is set from the velocity; what could you set it from instead?)
2. Spread shot: fire three fireballs, the outer two turned 15 degrees with `Quaternion.Euler(0, 0, 15) * direction`.
3. Aim with the gamepad's right stick: add a second binding to **Aim** and use the stick direction instead of the pointer when it's pushed.
4. Show a crosshair sprite that follows the pointer.